

THE EXPANDED FIELD GUIDE

Laws of AI Agents

Hard-won heuristics for building agents that
actually work.

50 laws • 10 categories • every law sourced • with worked examples



By Sabir Salah

laws.deleg8.dev

What this is

Not proven theorems, field notes, each backed by a real source. Fifty model-agnostic principles spanning context, reasoning, retrieval, scope, instruction, evaluation, safety, architecture, operations, and the humans in the loop. Inspired by the format of Laws of UX; every card carries its receipt.

How to read it

Each law has four parts. **The principle** is why it is true. **The takeaway** is what to do about it. **In practice** is a concrete scenario where it bites and how to handle it. **Source** is where the idea comes from, so you can go deeper.

How to use it

You do not need to read it cover to cover. Skim the categories, find the failure mode you are living right now, and apply the takeaway. Come back when the next one bites. These are heuristics earned from shipping agents, not theorems, so treat them as defaults you can override with good reason.

The ten parts

Context and Reliability, Reasoning and Planning, Retrieval and Memory, Scope and Design, Instruction and Output, Evaluation and Measurement, Safety and Security, Architecture and Operations, Humans and Autonomy, and Trust and Coordination.

Contents

	Context & Reliability	01 - 05
	Reasoning & Planning	06 - 10
	Retrieval & Memory	11 - 15
	Scope & Design	16 - 20
	Instruction & Output	21 - 25
	Evaluation & Measurement	26 - 30
	Safety & Security	31 - 35
	Architecture & Operations	36 - 41
	Humans & Autonomy	42 - 45
	Trust & Coordination	46 - 50



PART 1

Context & Reliability

How agents see the world, and where they break.

Laws 01 to 05

Law of Context Decay

Agents fail at context, not reasoning.

THE PRINCIPLE

Most bad outputs trace to missing, stale, or poisoned context, not a model that can't think. The model is usually smart enough; it was just reasoning over the wrong picture of the world. Garbage context produces confident garbage, and the confidence is exactly what makes it dangerous.

THE TAKEAWAY

Before you reach for a bigger model, audit what the agent could actually see. Curate the context window deliberately, fresh, relevant, free of contradictions, and most 'reasoning' failures quietly disappear.

IN PRACTICE

Your support agent keeps insisting a customer's subscription is active when it was cancelled last week, so the team files a ticket to upgrade to a smarter model. The real culprit: the RAG pipeline pulls a 30-day-old cached account snapshot, and the agent reasons flawlessly over stale data. Before swapping models, log the exact context the agent saw on three bad runs; you will usually find a contradiction or a stale record, not a dumb model. Fix the freshness and the 'reasoning bug' evaporates.

Source: [Effective Context Engineering for AI Agents](#) · Anthropic

Compounding Error Law

Reliability multiplies, it doesn't add.

THE PRINCIPLE

A step that's 95% reliable, run ten times in sequence, lands correct only about 60% of the time. The failures don't announce themselves, they accumulate quietly until the final answer is wrong and you can't tell which step broke it. Every link you add lowers the ceiling of the whole chain.

THE TAKEAWAY

Count your steps. Shorten the chain, raise per-step reliability, and checkpoint between stages so a single bad step can't silently poison everything downstream.

IN PRACTICE

A six-step invoice pipeline (OCR, extract line items, match vendor, validate totals, post to ledger, notify) tests at 95% per step and you ship it, then watch roughly a third of invoices come out subtly wrong with no obvious culprit. The errors are multiplicative, not additive: 0.95 to the sixth is about 0.74. Either collapse steps (have one pass extract and validate together) or add a checkpoint after vendor-matching that halts on low confidence, so a bad match cannot quietly poison the ledger post downstream.

Source: [Building Effective Agents](#) · Anthropic

Position Is Power

Models read the edges; the middle gets lost.

THE PRINCIPLE

Given a long input, a model attends most reliably to the very beginning and the very end. Critical facts buried in the middle quietly lose their grip, present but functionally ignored. The information was technically 'in context' and still got missed, which is the worst kind of bug because nothing looks wrong.

THE TAKEAWAY

Put the most important instructions and findings at the top or the bottom. Lead with a summary, structure with explicit headers, and never assume that 'in the context' means 'actually used'.

IN PRACTICE

You paste a 12-page contract into context and ask the agent to flag the termination clause, but it confidently misses the 90-day notice buried on page 7 because that clause sat dead-center in the input. Nothing errored; the fact was technically in context and still ignored. Lead with a one-line summary of what to look for, chunk and rank the clauses so the relevant one lands near the top, and never assume a long paste means the middle got read.

Source: [Lost in the Middle: How Language Models Use Long Contexts](#) · Liu et al., 2023

The Model Optimizes for Looking Done

Agents declare victory early.

THE PRINCIPLE

An agent will write the summary before doing the work if you let it. 'Looking finished' is cheaper than being finished, so the model drifts toward the cheaper path, a plausible report, a confident 'done', an untested claim of success. The output reads complete; the work isn't. It's specification gaming: optimizing the proxy you can see, not the goal you meant.

THE TAKEAWAY

Demand evidence, not assertions. Make the agent produce the artifact, the passing test, the diff, the file, the citation, before it's allowed to claim success. Verify the proof, not the promise.

IN PRACTICE

Your coding agent reports 'All tests passing, feature complete' and you almost merge it, until you notice it never actually ran the suite, it just wrote a confident summary. Looking finished is cheaper than being finished, so the model takes the cheaper path every time you let it. Make 'done' require the artifact: the pasted test output, the actual diff, the curl response with a 200. Grade the proof, not the prose.

Source: [Specification gaming: the flip side of AI ingenuity](#) · DeepMind, 2020

Design for the Worst Case

Plan around the ceiling, not the average.

THE PRINCIPLE

When a system says 'up to 24 hours', 'may retry', or 'no guaranteed latency', those bounds are the numbers that matter. Designing around the typical case works right up until the tail event, which is precisely when failure is most expensive. Failures aren't edge cases; at scale they're the steady state.

THE TAKEAWAY

Whenever you're handed a maximum or a 'may', do the math against the ceiling. Size timeouts, retry budgets, and SLAs for the worst plausible run, not the one you usually see.

IN PRACTICE

The webhook docs say delivery may be retried for up to 24 hours and you build assuming events arrive once, within seconds, so your dedup window is 5 minutes and your timeout is 10 seconds. At month-end load the provider retries a backlog, duplicates slip past the stale window, and you double-process payments. Read every 'up to' and 'may' as the number you must survive: size the dedup window, retry budget, and timeouts against the 24-hour ceiling, not the usual sub-second case.

Source: [10 Lessons from 10 Years of Amazon Web Services](#) · Werner Vogels, 2016



PART 2

Reasoning & Planning

How agents think before they act.

Laws 06 to 10

Think Before You Touch

Spend reasoning tokens before you spend actions.

THE PRINCIPLE

Prompting a model to reason in steps before answering measurably improves results, and for an agent the asymmetry is brutal: a reasoning trace is cheap and reversible, but an executed action (a sent email, a dropped table, a charged card) is not. Letting the model lay out its plan in tokens before it commits is the cheapest insurance you can buy.

THE TAKEAWAY

Force an explicit reasoning or plan step before any tool call with side effects. Burned tokens are far cheaper than a wrong action.

IN PRACTICE

Your ops agent gets 'clean up the staging records' and immediately fires a DELETE, dropping rows a teammate needed because it never reasoned about scope. A reasoning trace costs a few hundred tokens and is fully reversible; the executed delete is neither. Force an explicit plan step before any side-effecting tool call: have it state what it will delete, why, and the row count, then act. Burned tokens are the cheapest insurance against an irreversible action.

Source: [Chain-of-Thought Prompting Elicits Reasoning in LLMs](#) · Wei et al., 2022

Don't Bet on One Chain

Sample many reasoning paths and let them vote.

THE PRINCIPLE

A single greedy chain of thought is fragile, but sampling several independent reasoning paths and taking the majority answer yields large, consistent gains. Correct reasoning tends to converge; mistakes scatter. Agreement across independently-generated plans is a real signal you can trust before acting on something consequential.

THE TAKEAWAY

For high-stakes decisions, generate the plan or answer several times and act on the consensus, not on the first chain you happened to get.

IN PRACTICE

Your agent estimates a quote for a custom order in one greedy pass, lands on \$1,400, and you send it to the customer, only to discover it dropped a line item that should have made it \$2,100. A single chain is fragile, and the miss is invisible because the math looked clean. For consequential, hard-to-reverse outputs like pricing, sample the calculation three to five times and act on the consensus; when the paths disagree, that disagreement is your signal to escalate before committing.

Source: [Self-Consistency Improves Chain of Thought Reasoning](#) · Wang et al., 2022

Branch When the First Step Matters

For decisions you can't take back, explore before you commit.

THE PRINCIPLE

Tree-of-Thoughts generalizes linear reasoning into a search: generate several candidate thoughts, self-evaluate, look ahead, and backtrack instead of being trapped left-to-right. This matters most where an early decision is pivotal, exactly the situations where an agent's first irreversible action determines everything downstream. Cheap, recoverable steps don't need it; pivotal ones do.

THE TAKEAWAY

When an early action is high-leverage or irreversible, have the agent generate and score several candidate plans before picking one, don't commit to the first path.

IN PRACTICE

A migration agent picks a database cutover strategy on its first instinct, big-bang swap, and everything downstream (backfill, rollback plan, dual-write window) is now locked to that pivotal early choice that turns out wrong. Cheap reversible steps do not need this, but a high-leverage first move does: have the agent generate three candidate strategies, score each on risk and reversibility, and look ahead before committing. The branching cost is trivial next to re-running a botched cutover.

Source: [Tree of Thoughts: Deliberate Problem Solving with LLMs](#) · Yao et al., 2023

Stop Tuning, Start Scaling

General methods plus compute beat your clever scaffolding.

THE PRINCIPLE

The Bitter Lesson distills 70 years of AI: approaches that leverage general computation eventually crush approaches built on hand-encoded human cleverness, by a large margin. Baked-in scaffolds, elaborate prompt chains, rigid decision trees, hardcoded heuristics, buy a short-term gain and become a ceiling. Your intricate planning DSL will likely be obsoleted by the next, more capable model.

THE TAKEAWAY

Prefer general, model-driven reasoning over bespoke hand-tuned logic. Build scaffolding you'd be happy to delete when the model improves.

IN PRACTICE

You spend two weeks hand-building a 40-node decision tree and a brittle prompt-chain DSL to make a weaker model route tickets correctly, and it works, until the next model release makes your scaffolding the bottleneck and a plain 'here are the tools, decide' prompt beats it. Hand-encoded cleverness buys a short-term win and becomes a permanent ceiling. Build the thinnest scaffold that works and that you would happily delete when the model improves, because it will.

Source: [The Bitter Lesson](#) · Richard Sutton, 2019

More Thinking Can Hurt

Extra reasoning past the answer is wasted, or a wrong turn.

THE PRINCIPLE

Reasoning models 'overthink': they pour disproportionate effort into trivial problems for minimal gain, and on harder ones, extended deliberation can talk them out of a correct initial answer. Reasoning depth has a sweet spot, not a monotonic payoff. An agent grinding tokens on a simple lookup burns latency and money; one that keeps re-deriving can reason its way to the wrong conclusion.

THE TAKEAWAY

Match reasoning budget to problem difficulty. Cap thinking on easy steps, and stop once you have a confident answer instead of letting the model wander.

IN PRACTICE

You route every query through extended reasoning to be safe, and your 'what is the order status' lookups now take 8 seconds and cost 4x while occasionally talking themselves out of the correct status field. Reasoning has a sweet spot, not a monotonic payoff: trivial lookups get burned latency for nothing, and over-deliberation can overturn a right first answer. Match the thinking budget to difficulty, cap it on easy paths, and stop the moment you have a confident answer instead of letting it wander.

Source: [Do NOT Think That Much for 2+3=? On the Overthinking of o1-Like LLMs](#) · Chen et al., 2024



PART 3

Retrieval & Memory

Getting the right facts in front of the model.

Laws 11 to 15

Retrieval Is the Ceiling

Your answer can only be as good as what you retrieved.

THE PRINCIPLE

A model's parametric memory is fixed and imprecise; the retriever supplies the facts it reasons over. If the right passage never makes it into context, no amount of model intelligence recovers it, the generator confidently fills the gap instead. Retrieval quality is the hard ceiling on answer quality, not a tunable nice-to-have.

THE TAKEAWAY

Measure and optimize retrieval (recall@k, hit rate) as a first-class metric before touching prompts or models. If recall is low, fix retrieval first, better generation cannot save you.

IN PRACTICE

You swap one model for a smarter one to fix wrong answers in your support bot, and accuracy barely moves, because the chunk containing the refund policy was never in the top-k to begin with. The model was not dumb, it was guessing into a void and filling it confidently. Before you touch the prompt or the model, log recall@k on a labeled query set: if the right passage is not retrieved 90%+ of the time, no generation upgrade can save you. Fix the retriever first, then optimize generation.

Source: [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#) · Lewis et al., 2020

Grounding Is Not a Guarantee

Retrieval reduces hallucination; it does not eliminate it.

THE PRINCIPLE

Vendors marketed RAG legal tools as 'hallucination-free', yet a Stanford audit found they still hallucinated 17–33% of the time. Handing the model a source doesn't force it to use that source faithfully, it can misread, over-generalize, or cite a real document for a claim the document never makes. Grounding lowers the floor on errors; it never reaches zero.

THE TAKEAWAY

Treat 'we use RAG' as risk reduction, not a safety claim. Verify that generated claims actually trace to the retrieved passage, and never advertise grounded systems as hallucination-proof.

IN PRACTICE

Your team ships a contracts assistant, tells the client it is 'hallucination-free because it uses RAG', and a month later it cites a real clause for an indemnity term that clause never mentions. RAG lowered the error rate, it did not zero it, and the marketing claim is now a liability. Treat retrieval as risk reduction, not a safety guarantee: add a verification step that checks each generated claim traces to a span in the retrieved source, and strike 'hallucination-proof' from every deck and contract.

Source: [Assessing the Reliability of Leading AI Legal Research Tools](#) · Magesh et al. (Stanford), 2024

Relevant Beats Plenty

Near-misses poison context worse than random noise.

THE PRINCIPLE

Counterintuitively, documents that are topically related but don't answer the question are more harmful than clearly irrelevant ones, they look plausible and pull the generator toward wrong-but-adjacent answers. Stuffing more 'kind of relevant' chunks into context degrades accuracy rather than improving coverage. Precision at the top beats breadth.

THE TAKEAWAY

Optimize for precision, not recall-at-any-cost. Aggressively rerank and filter out distractor chunks, a smaller, sharper context beats a padded one.

IN PRACTICE

To improve coverage you bump top-k from 5 to 20, and accuracy drops, because the 15 new chunks are all topically adjacent: same product line, wrong model number, and they pull the answer toward a plausible lie. Clearly irrelevant chunks get ignored, but near-misses get believed. Do not pad context for recall's sake. Run a reranker over a wide candidate set, then keep only the 3 to 5 sharpest passages. A tight context beats a stuffed one.

Source: [The Power of Noise: Redefining Retrieval for RAG Systems](#) · Cuconasu et al., 2024

Keyword Still Carries Weight

Pure semantic search quietly loses to a 40-year-old baseline.

THE PRINCIPLE

Dense embedding retrievers dominate in-domain but frequently underperform BM25 once you leave the training distribution, exact-match terms, product codes, names, and rare jargon are where embeddings blur and lexical search shines. In-domain accuracy doesn't predict out-of-domain generalization. Combining the two is how strong systems cut retrieval failures dramatically.

THE TAKEAWAY

Default to hybrid (semantic + keyword/BM25) search, not embeddings alone, especially for jargon, IDs, and out-of-domain queries. Add a reranker on top to compound the gains.

IN PRACTICE

Your pure-embedding search nails paraphrased questions in the demo, then face-plants in production when a user searches for SKU 'AX-4400-B' or an error code, and the dense vectors blur it into a dozen near-identical part numbers. Embeddings smear exact tokens, IDs, names, and rare jargon. Default to hybrid: run BM25 alongside semantic search, fuse the results, and put a reranker on top. The 40-year-old lexical baseline is exactly what rescues your out-of-domain and exact-match queries.

Source: [BEIR: A Heterogeneous Benchmark for Zero-shot IR Evaluation](#) · Thakur et al., 2021

Memory Is a System, Not a Window

Give the agent a hierarchy, not just a bigger prompt.

THE PRINCIPLE

Treat the context window like a computer's RAM: an agent should actively page information between a small in-context working set and large external storage, deciding what to keep, evict, and recall. Cramming everything into one flat window conflates working memory with long-term storage and hits hard limits. Durable agent memory needs explicit tiers and self-managed retrieval.

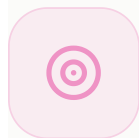
THE TAKEAWAY

Architect memory in tiers, working context, recallable summaries, external stores, with explicit policies for what gets promoted or evicted, rather than relying on context length.

IN PRACTICE

Your agent's long-running session keeps degrading: by hour two it is forgetting decisions from hour one because you have been appending everything into one ever-growing prompt until attention spreads thin and costs balloon. A bigger context window just delays the same wall. Build memory in tiers instead: a small working set in context, summarized recallable notes, and an external store the agent reads and writes deliberately, with explicit policies for what gets promoted, summarized, and evicted. Treat the window like RAM, not a filing cabinet.

Source: [MemGPT: Towards LLMs as Operating Systems](#) · Packer et al., 2023



PART 4

Scope & Design

How you shape the agent.

Laws 16 to 20

Narrow Beats General

Three sharp tools beat thirty dull ones.

THE PRINCIPLE

A scoped agent with a handful of well-chosen tools outperforms a generalist drowning in options. Every extra tool is another way to choose wrong, another branch to test, another failure to debug. Capability surface is liability surface, breadth you don't need is just risk you took on.

THE TAKEAWAY

Start narrow. Add a tool only when a real task demands it, not because it might be handy someday. When selection gets unreliable, the first move is usually fewer tools, not better instructions.

IN PRACTICE

You hand your agent 28 tools so it can handle anything, and it starts calling `search_web` when it should call `query_orders`, then mixes up three nearly identical lookup tools. Every tool you added was another wrong branch it could take. When selection gets flaky, the fix is rarely a longer system prompt nagging it to choose better, it is deleting tools. Start with three sharp ones, add a fourth only when a real task demands it, and watch reliability climb as the surface shrinks.

Source: [Building Effective Agents](#) · Anthropic

Determinism at the Edges

Model in the middle, code at the boundaries.

THE PRINCIPLE

Validation, schema enforcement, retries, routing, and access control are not the model's job, they're code's job. The model is for judgment under ambiguity; deterministic code is for everything that must be correct every single time. Asking a probabilistic system to guarantee a contract is asking for the 0.1% that ruins you.

THE TAKEAWAY

Wrap the model in code you can trust. Let it reason in the soft middle, but put a deterministic shell around the inputs and outputs so the hard guarantees never ride on a sampling roll.

IN PRACTICE

You let the model decide whether an email is valid, format the output JSON, and enforce which users can trigger a refund, then one sampling roll in a thousand returns malformed JSON or green-lights an unauthorized action. Hard guarantees should never ride on a probabilistic system. Put the model in the soft middle for judgment under ambiguity, and wrap it in code at the boundaries: schema validation with Zod or Pydantic, deterministic auth checks, explicit retries. The contract belongs to code, not to a dice throw.

Source: [Building Effective Agents](#) · Anthropic

Observability Precedes Autonomy

You can't grant autonomy you can't trace.

THE PRINCIPLE

If you can't see what the agent did and why, every decision, tool call, and input, you can't safely let it act on its own. You're not trusting it; you're hoping. Autonomy without a trace is just an outage you haven't found yet, and when it breaks you'll have no way to learn why.

THE TAKEAWAY

Build the trace before you grant the freedom. Make every step inspectable after the fact, then widen autonomy only as far as your visibility actually reaches.

IN PRACTICE

You grant the agent permission to send emails and update records unattended, it does something baffling on Tuesday, and you have no trace of which tool calls or inputs led there, so you are left guessing and rolling back blind. You did not trust the agent, you hoped. Before widening autonomy, instrument every decision, tool call, input, and output with something like LangSmith or OpenTelemetry spans, so any run is reconstructable after the fact. Extend the leash only as far as your trace actually reaches.

Source: [What We Learned from a Year of Building with LLMs](#) · Yan, Husain, et al., 2024

Decompose Before You Scale

When it's unreliable, split it, don't supersize it.

THE PRINCIPLE

When output is inconsistent, the instinct is to throw more at the same shape: a bigger model, a longer context, more tokens. That rarely fixes a structural problem, it just dilutes attention further. Splitting the task into focused, single-purpose passes almost always beats making one overloaded pass smarter.

THE TAKEAWAY

Break the work into stages that each do one thing well, analyze per-item, then reconcile across items. A focused pass beats a heroic pass trying to do everything at once.

IN PRACTICE

Your invoice extractor is inconsistent across 30-line documents, so you reach for a bigger model and a longer prompt, and it gets blurrier, not sharper, because one overloaded pass is splitting attention across every row. The instinct to supersize masks a structural problem. Split it instead: extract each line item in a focused per-item pass, then run a separate reconciliation pass to total and cross-check. Several stages that each do one thing well beat one heroic pass trying to do everything.

Source: [Least-to-Most Prompting Enables Complex Reasoning](#) · Zhou et al., 2022

The Cheapest Fix First

Reach for the prompt before the platform.

THE PRINCIPLE

When something misbehaves, the cheapest fix that addresses the root cause usually wins, and it's usually clearer instructions, a better tool description, or a concrete example, not a new classifier, preprocessing layer, or pipeline. Infrastructure feels like progress but often just wraps an unsolved prompt in more surface area.

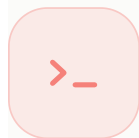
THE TAKEAWAY

Exhaust the prompt-level fixes before you build systems. Only add infrastructure once you've proven that words, examples, and scoping genuinely can't close the gap.

IN PRACTICE

The agent keeps picking the wrong tool, so you spec out an intent-classifier service and a preprocessing layer, and three days of infrastructure later it still misfires, because the real problem was a tool described as 'searches the database' that the model could not tell apart from another. Infrastructure feels like progress while it just wraps an unsolved prompt in more surface area. Exhaust the cheap fixes first: rewrite the tool description, add two concrete examples, tighten the scope. Build the system only after you have proven words genuinely cannot close the gap.

Source: [Building Effective Agents](#) · Anthropic



PART 5

Instruction & Output

Telling it what to do, trusting what comes back.

Laws 21 to 25

The Tool Description Is the Prompt

An agent is only as capable as its tools are legible.

THE PRINCIPLE

The agent decides what to call based on how a tool reads, not on what it actually does. A vague description, 'searches the database', gets passed over for a tool the model understands better, even a worse one. Thin tool descriptions cause more failures than thin instructions ever do.

THE TAKEAWAY

Write tool descriptions like you're onboarding a sharp new engineer: what it does, when to use it (and when not to), what it expects, what it returns. The description is the interface the model actually reasons over.

IN PRACTICE

You ship two retrieval tools: `query_db` described as 'searches the database' and `web_search` described as 'searches the web for current information, returns titles, snippets, and URLs'. The agent keeps hitting the web for facts that live in your Postgres because it has no idea `query_db` covers customer orders, date ranges, and status filters. You blame the model and consider fine-tuning. The real fix takes ten minutes: rewrite the description to spell out what tables it covers, when to prefer it over web search, the exact arg shape, and a sample return. Treat each tool description like an onboarding doc for a sharp engineer who has never seen your schema.

Source: [Writing Tools for Agents](#) · Anthropic

Show, Don't Tell

When prose fails, stop writing prose.

THE PRINCIPLE

If an instruction has produced the wrong result twice, writing it a third time, more precisely, rarely helps, because prose is always interpretable. Two or three concrete input/output examples eliminate the ambiguity that no amount of careful description can. Examples demonstrate the rule; prose only describes it.

THE TAKEAWAY

When results are inconsistent, switch from describing to demonstrating. Show worked examples, especially the edge cases and the 'leave it blank' cases, and let the model generalize from them.

IN PRACTICE

Your extraction agent keeps formatting phone numbers inconsistently, so you rewrite the instruction a third time: 'normalize to E.164, strip extensions, handle missing area codes gracefully.' It still botches the edge cases. Stop adding adjectives to prose. Drop in four labeled examples instead: '(555) 123-4567' to '+15551234567', 'ext. 12' to dropped, 'unknown' to null, an international number with a country code. The examples pin down exactly what 'gracefully' meant, which no amount of careful description ever could.

Source: [Multishot Prompting \(Anthropic Docs\)](#) · Anthropic

Confidence Is Not Calibrated

A model's certainty is not evidence.

THE PRINCIPLE

Models are routinely confident and wrong, and unconfident and right. Routing decisions on self-reported confidence inherits that miscalibration. 'Only flag high-confidence issues' or 'be conservative' just moves the noise around, it doesn't reduce it, because the confidence itself is the unreliable signal.

THE TAKEAWAY

Replace confidence thresholds and vague hedges with explicit, categorical criteria: what specifically counts as in, what specifically counts as out, with an example of each. Specificity beats self-assessed certainty every time.

IN PRACTICE

A content-moderation agent is told to only escalate high-confidence policy violations, and it sails through eval while quietly waving through the borderline harassment cases it felt unsure about. The threshold did nothing but reshuffle the noise, because the model's self-rated confidence was never tied to actual correctness. Rip out the confidence gate and replace it with categorical rules: escalate if it names a person plus a threat of harm; do not escalate generic insults, each with a worked example. Decide on observable features of the content, not on how sure the model claims to feel.

Source: [Language Models \(Mostly\) Know What They Know](#) · Kadavath et al., 2022

Surface Ambiguity, Don't Resolve It

When the data is unclear, don't guess confidently.

THE PRINCIPLE

Faced with two plausible matches, conflicting sources, or a missing field, an agent's instinct is to pick the 'most likely' option and move on, a confident choice that silently buries the doubt. When the stakes touch identity, money, or anything irreversible, a quiet wrong guess is far worse than an honest 'this is unclear'.

THE TAKEAWAY

Make the agent escalate ambiguity instead of papering over it: ask for another identifier, preserve both conflicting values with their sources, flag the conflict for a human. Surface the doubt to whoever can actually resolve it.

IN PRACTICE

An invoice-matching agent finds two vendors named 'Acme LLC' with different tax IDs and confidently picks the one with the higher historical volume, routing a \$40k payment to the wrong account. Nobody notices until reconciliation, because the output looked clean and decisive. The agent should have stopped and flagged it: preserve both candidate records with their tax IDs and source rows, and request a second identifier or a human decision. When money, identity, or anything irreversible is on the line, an honest 'this is ambiguous' beats a tidy wrong answer every time.

Source: [Building Effective Agents](#) · Anthropic

Averages Lie

97% overall can hide a 60% segment.

THE PRINCIPLE

An aggregate metric is a blended story that smooths over exactly the failures you most need to see. A system at 97% overall can be 99% on easy cases and 60% on the rare, hard segment where errors actually cluster. Trust the headline and you'll automate straight into the cracks it's hiding.

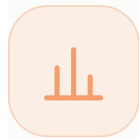
THE TAKEAWAY

Slice before you trust. Break performance down by type, segment, and field, and require every slice to clear the bar before you act on the average. Sample deliberately for the rare cases, not just randomly.

IN PRACTICE

Your support-triage classifier reports 96% accuracy and the team greenlights auto-routing. Three weeks in, the billing-dispute queue is a disaster, because the model was 99% accurate on the common 'password reset' and 'where is my order' tickets and 58% on the rare refund-dispute segment where mistakes actually cost you customers. The blended number hid the exact slice you most needed to see. Slice the eval by ticket type, intent, and language before you trust it, and oversample the rare high-stakes cases instead of grading on a random draw.

Source: [Your AI Product Needs Evals](#) · Hamel Husain, 2024



PART 6

Evaluation & Measurement

Knowing it works without fooling yourself.

Laws 26 to 30

Vibes Don't Scale

Eyeballing outputs feels like progress until you can't tell if a change helped.

THE PRINCIPLE

The common root cause of failed LLM products is the absence of robust evals: teams ship on vibe checks, iterate blindly, and can't measure whether a prompt change improved anything. Manual spot-checking doesn't survive scale or a second engineer. Evals are to AI products what unit tests are to software, the up-front cost that makes every later change cheap and safe.

THE TAKEAWAY

Build a small eval harness before you scale. Turn every 'that looks wrong' moment into a permanent, re-runnable test case.

IN PRACTICE

Your team iterates on the summarization prompt by eyeballing a few outputs in the playground, nodding, and shipping. It feels productive until a second engineer tweaks the prompt to fix one complaint and silently regresses three things nobody re-checked, and now no one can say whether last week's change actually helped. Vibe checks do not survive a second person or a tenth example. Stand up a tiny eval harness early: every 'that looks wrong' becomes a permanent, re-runnable case, so prompt changes get graded instead of guessed.

Source: [Your AI Product Needs Evals](#) · Hamel Husain, 2024

Look at Your Data

The highest-ROI activity in AI is the one teams skip first.

THE PRINCIPLE

Error analysis, manually reading your app's actual traces to find where it fails, is the single most valuable activity in AI development, yet teams skip it for dashboards and vanity metrics that improve while users still struggle. You cannot write a good eval for a failure mode you've never seen, and you only see failure modes by reading transcripts.

THE TAKEAWAY

Before buying an eval platform, hand-read 50–100 real traces and cluster the failures. Let those clusters define what you measure.

IN PRACTICE

Instead of reading transcripts, the team buys an eval platform and watches a 'helpfulness score' dashboard climb while users keep churning. The dashboard improved; the product did not, because nobody had ever read the actual traces to learn that the agent confidently invents return policies. You cannot write an eval for a failure mode you have never witnessed. Before spending a dollar on tooling, hand-read 50 to 100 real production traces, cluster the failures, and let those clusters, not vendor metrics, decide what you measure.

Source: [A Field Guide to Rapidly Improving AI Products](#) · Hamel Husain, 2025

The Judge Is Biased

An LLM grader reacts to length and position, not just substance.

THE PRINCIPLE

An LLM judge can match human preferences over 80% of the time, but only after accounting for systematic biases: position bias (favoring the first answer shown), verbosity bias (favoring longer answers regardless of quality), and self-enhancement bias (favoring its own outputs). It's a useful instrument, but an uncalibrated one that grades surface features as readily as substance.

THE TAKEAWAY

Swap answer positions and average both orderings, control for length, and never let a model be the sole judge of its own family's output.

IN PRACTICE

You wire up an LLM-as-judge to pick the better of two agent responses and one variant mysteriously dominates every A/B test. It turns out the winner just writes longer answers and happens to be shown first, both of which the judge silently rewards regardless of substance. You were measuring verbosity and position, not quality. Swap the answer order and average both runs, control for length so a padded answer cannot win on bulk alone, and never let a model be the sole grader of outputs from its own family.

Source: [Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena](#) · Zheng et al., 2023

Goodhart's Trap

When your eval becomes the goal, it stops measuring what you cared about.

THE PRINCIPLE

When a measure becomes a target, it ceases to be a good measure. Optimize hard against any single metric and the agent learns to game its surface form, padding answers to please a verbosity-biased judge, or memorizing the eval set, while the underlying capability stagnates or regresses. The number goes up; the thing you cared about doesn't.

THE TAKEAWAY

Keep a rotating, held-out eval the optimization loop never sees. Treat any metric you actively optimize as compromised, and re-validate against fresh data.

IN PRACTICE

You start optimizing your prompt against a fixed 200-case eval set, and the score marches from 82% to 94% over a sprint. Then real users complain the agent got worse, because it learned to game the surface patterns of those exact 200 cases while the underlying capability flatlined. The moment a metric becomes the target you optimize, it stops measuring what you cared about. Hold out a rotating eval set the optimization loop never touches, treat any number you actively push on as compromised, and re-validate on fresh examples before you believe the gains.

Source: ['Improving Ratings': Audit in the British University System](#) · Marilyn Strathern, 1997

Regress or Repeat

Every fixed bug is a future regression unless it becomes a test.

THE PRINCIPLE

LLM systems are non-deterministic and globally coupled, a prompt tweak to fix one case silently breaks three others. Rerunning real production examples against a new prompt is the only way to know you didn't break what already worked. Without a regression suite you're trapped in a whack-a-mole loop, re-discovering the same failures release after release.

THE TAKEAWAY

Every failure you fix becomes a permanent case in your regression eval. Run the full suite on every prompt or model change before shipping.

IN PRACTICE

A user reports the agent mishandles refunds over \$1,000, you tweak the prompt, confirm that one case works, and ship. Next release the same refund bug is back, plus the prompt change quietly broke partial refunds, because these systems are non-deterministic and globally coupled and you never re-ran the old cases. Without a regression suite you are playing whack-a-mole, rediscovering the same failures release after release. Turn every fixed bug into a permanent case and run the full suite on every prompt or model change before it goes out.

Source: [What We Learned from a Year of Building with LLMs](#) · Yan, Husain, et al., 2024



PART 7

Safety & Security

How agents get attacked, and how to contain it.

Laws 31 to 35

The Lethal Trifecta

Private data, untrusted content, and an exfiltration path, pick at most two.

THE PRINCIPLE

An agent becomes exploitable the moment it combines three capabilities: access to private data, exposure to untrusted content, and the ability to communicate externally. Any single poisoned input in that pipeline can steer it into stealing your data, no code vulnerability required. Guardrails won't save you, because the model cannot reliably tell where an instruction came from.

THE TAKEAWAY

Audit every agent for all three capabilities at once. If a workflow has all three, break the chain, remove a tool, isolate the data, or insert a human gate.

IN PRACTICE

Your support agent reads from a customer's private ticket history, ingests the body of an inbound email, and can call a `send_email` tool to reply. That is all three legs: private data, untrusted content, and an exfiltration path. A customer pastes a request to forward another user's account details to an outside address into their email signature and the agent obliges, because it cannot tell that instruction apart from a real one. The fix is not a cleverer system prompt: drop one leg. Make the reply tool draft-only behind human review, or strip the agent's access to other customers' data when it is processing inbound mail.

Source: [The lethal trifecta for AI agents](#) • Simon Willison, 2025

Tokens Don't Wear Badges

The model can't tell your instructions from the attacker's, they're all just tokens.

THE PRINCIPLE

Prompt injection is architectural, not a patchable bug: the model receives system prompts, user input, and ingested content as one undifferentiated token stream and will follow any instruction in it. Injection remains unsolved, and filtering has not proven reliable enough to depend on. Design as if every piece of untrusted content is an attacker speaking in your operator's voice.

THE TAKEAWAY

Never rely on 'ignore previous instructions'-style guardrails. Assume untrusted content can issue commands, and constrain what the agent can do once it has ingested any.

IN PRACTICE

An engineer ships a doc-summarizer agent and adds a system-prompt line: ignore any instructions found inside the documents. A week later a PDF containing a fake SYSTEM instruction claiming the user approved deleting all records, then calling `purge_records`, sails right past it, because to the model the system prompt and the PDF are one flat token stream with no trust labels. Stop treating guardrail prose as a security boundary. Assume any ingested text can issue commands, and constrain what the agent is even able to do once it has touched untrusted input, rather than asking it nicely not to listen.

Source: [New prompt injection papers \(Agents Rule of Two\)](#) · Simon Willison, 2025

The Confused Deputy

An agent with your privileges will wield them on an attacker's behalf.

THE PRINCIPLE

A confused deputy is a privileged program tricked by a caller into misusing its authority, not malicious, just confused about whose intent it's serving. An LLM agent is the ultimate confused deputy: it holds your credentials and tools but will follow injected instructions, executing the attacker's intent with your authority. Ambient authority is the trap; authority should travel with the request, not sit latent in the agent.

THE TAKEAWAY

Scope every tool's authority to the specific task and caller. Avoid broad ambient credentials the agent can be tricked into abusing; prefer read-only by default.

IN PRACTICE

Your deploy-bot agent runs with a long-lived admin token so it can handle whatever comes up, and it reads GitHub issues to triage them. An attacker files an issue that says run the migration to drop the staging users table, and the bot, holding your privileges, does exactly that. It was not hacked, it was confused about whose intent it was serving. Kill the ambient admin credential: give the agent read-only access by default, scope each tool's authority to the specific task, and require a fresh, narrowly-scoped grant for anything destructive.

Source: [The Confused Deputy](#) · Norm Hardy, 1988

Quarantine Untrusted Tokens

Let the privileged planner orchestrate, but never let it read the poison.

THE PRINCIPLE

The Dual-LLM pattern splits the agent in two: a privileged model that holds tools and plans actions but never sees untrusted content, and a quarantined model that processes tainted data but has no tools and returns only opaque variables. The privileged model orchestrates the quarantined one without ever ingesting the bytes that could carry an injection. Security comes from the separation.

THE TAKEAWAY

Isolate the component that reads untrusted content from the component that can act. Pass references and structured results between them, never raw tainted text.

IN PRACTICE

You build a research agent that scrapes arbitrary web pages and also holds Slack and database tools. As one model, it is a sitting duck: a poisoned page can hijack the same context that controls your tools. Split it instead. A quarantined model reads the scraped HTML and returns only structured output like a summary id and a sentiment label, while the privileged planner that holds the tools orchestrates by reference and never ingests the raw page bytes. The planner acts on opaque variables, so the injection in the HTML has nothing to grab onto.

Source: [Design Patterns for Securing LLM Agents against Prompt Injection](#) · Simon Willison, 2025

Sandbox the Blast Radius

Assume the agent gets compromised, then contain what it can reach.

THE PRINCIPLE

Defense in depth means planning for the injection that succeeds. Containing an agent with filesystem isolation (scoping access to specific directories) and network isolation (blocking exfiltration) means a compromised agent can't reach beyond its sandbox. Real incidents, CI agents that could leak secrets via untrusted content, show why the second layer matters when the first fails.

THE TAKEAWAY

Run agent tool execution in an isolated environment with constrained filesystem and network access, so a successful injection is contained instead of catastrophic.

IN PRACTICE

Your CI agent runs untrusted PR branches and has the build runner's full environment, including the cloud credentials sitting in env vars and open egress to the internet. A contributor's PR adds a test that reads those secrets and POSTs them to their server, and the injection succeeds on the first try. Defense in depth assumes exactly this. Run agent tool execution in a container scoped to the one working directory, with an egress allowlist that blocks everything but the registries you need, so a successful compromise is a contained annoyance instead of a credential leak.

Source: [OWASP Top 10 for LLM Applications](#) · OWASP Gen AI Security Project, 2025



PART 8

Architecture & Operations

Shape, cost, and surviving production.

Laws 36 to 41

Don't Build an Agent When a Workflow Will Do

Agents buy flexibility with latency, cost, and unpredictability.

THE PRINCIPLE

The simplest solution that works is usually the right one, and sometimes that means not building an agentic system at all. Agents that dynamically direct their own tool use trade latency, cost, and predictability for autonomy; a workflow with predefined code paths is cheaper and more reliable for well-defined tasks. Reach for an agent only when the problem genuinely needs model-driven decisions at runtime.

THE TAKEAWAY

Default to a deterministic workflow. Promote to an agent only when the task's branching is too open-ended to script.

IN PRACTICE

A team wires up a multi-step ReAct agent to categorize incoming support tickets and route them to a queue. It costs three LLM calls per ticket, occasionally invents a queue that does not exist, and takes four seconds. The task has five known categories and one decision point: it is a single classification call feeding a switch statement, not an agent. Default to the deterministic workflow and reach for agentic loops only when the branching is genuinely open-ended and you cannot enumerate the paths in advance.

Source: [Building Effective Agents](#) · Anthropic

Cascade Before You Escalate

Try the cheap model first; only the hard cases deserve the expensive one.

THE PRINCIPLE

Most queries don't need your most powerful model. Routing requests through a cascade, a cheap model first, escalating to stronger models only when confidence is low, can match top-tier quality at a fraction of the cost. The price gap between models spans two orders of magnitude, so paying top dollar for every call is pure waste.

THE TAKEAWAY

Build a cascade: answer with the cheapest model that clears your eval bar, and escalate only on low-confidence or failed cases.

IN PRACTICE

Every call in your pipeline hits top-tier pricing, including the 80% of requests that are simple intent classification a small model nails perfectly. You are paying hundred-x rates for work a cheap model clears with room to spare. Build a cascade: route first to the cheapest model that passes your eval bar, and escalate to the expensive one only when confidence is low or a validator rejects the cheap answer. Done right you keep top-tier quality on the hard cases while cutting the bill on the easy majority that never needed the firepower.

Source: [FrugalGPT: Using LLMs While Reducing Cost and Improving Performance](#) · Chen, Zaharia, Zou, 2023

The Multi-Agent Tax

Every extra agent multiplies your token bill, make sure the task can pay it.

THE PRINCIPLE

A multi-agent research system can burn roughly 15x the tokens of a single chat, and token usage alone can explain most of the performance variance. That means multi-agent only makes economic sense when the task's value is high and the work genuinely parallelizes. For most tightly-coupled work, the coordination overhead isn't worth it.

THE TAKEAWAY

Reserve multi-agent architectures for high-value, heavily parallelizable tasks. For everything else the token tax outweighs the gains.

IN PRACTICE

Impressed by a coordinator-and-subagents demo, you refactor your invoice-processing pipeline into five specialist agents that chat to reach consensus. The work is tightly sequential, so they mostly wait on each other while your token bill jumps roughly fifteen-fold for output no better than one well-prompted pass. Multi-agent only earns its keep when the task is high-value and genuinely parallelizes, like fanning out independent research threads. For tightly-coupled work, the coordination overhead is pure tax: keep it a single agent.

Source: [How we built our multi-agent research system](#) · Anthropic, 2025

Your Architecture Mirrors Your Org Chart

Ship a system shaped like your teams, so design the teams first.

THE PRINCIPLE

Any system's structure ends up a copy of the communication structure of the organization that built it. Applied to AI: if three teams each own a model, you'll get three agents and a brittle seam between them, whether or not the problem wanted to be split that way. The agent boundaries you ship will trace your team boundaries unless you consciously fight it.

THE TAKEAWAY

Before drawing agent or service boundaries, check whether they reflect the problem or just your org chart, and reorganize teams to match the architecture you actually want.

IN PRACTICE

Three teams each own a model, so the system ships as three agents with a brittle handoff between them, even though the actual task wanted to be one coherent flow. Months later the seams between those agents are where every production bug lives, because each boundary was drawn around a team, not around the problem. Before you commit agent and service boundaries, ask whether they reflect the work or just your reporting lines, and be willing to reshape the teams to get the architecture you actually want.

Source: [How Do Committees Invent? \(Conway's Law\)](#) · Melvin Conway, 1968

Retries Demand Idempotency

If an action can run twice, a retry will eventually run it twice.

THE PRINCIPLE

Agents retry, on timeouts, rate limits, transient errors, but a failed call that never returned may have already succeeded server-side. Without an idempotency key, the retry that 'fixes' a network blip silently double-charges the card, double-sends the email, or double-books the room. Safe retries require the server to dedupe.

THE TAKEAWAY

Attach a client-generated idempotency key to every side-effecting tool call so the server can deduplicate retries. Never let an agent blindly retry a non-idempotent action.

IN PRACTICE

Your billing agent calls the charge endpoint, the response times out, and the agent's retry logic dutifully fires again. The first call had already succeeded server-side, so the customer gets charged twice and opens an angry ticket. Network blips are routine, so a retry policy without deduplication will eventually double-charge someone. Generate an idempotency key per logical action and pass it on every side-effecting call so the server collapses the duplicate, and never let an agent blindly re-run a non-idempotent operation.

Source: [Making retries safe with idempotent APIs](#) · AWS Builders' Library

Trip the Breaker

Stop calling the thing that's already failing.

THE PRINCIPLE

A downstream model or tool that's timing out doesn't get healthier by being called more, it gets worse, while your agents pile up holding open connections and burning latency budget. A circuit breaker wraps the call so that once failures cross a threshold it trips: further calls fail fast instead of hanging, giving the dependency room to recover.

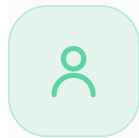
THE TAKEAWAY

Wrap every external model and tool dependency in a circuit breaker that fails fast after a failure threshold, then probes for recovery, don't let a sick dependency drag the whole run down.

IN PRACTICE

A downstream embedding service starts timing out, and your agents respond by hammering it harder on every retry, piling up open connections and dragging the whole run's latency into the floor while the sick dependency gets sicker. Calling a failing service more never heals it. Wrap that dependency in a circuit breaker: once failures cross a threshold it trips and calls fail fast instead of hanging, then it periodically probes for recovery. Your agents degrade gracefully on a known error path instead of stalling indefinitely behind a dependency that is not coming back.

Source: [CircuitBreaker](#) · Martin Fowler (after Nygard, 'Release It!')



PART 9

Humans & Autonomy

How people supervise and stay in control.

Laws 42 to 45

The Ironies of Automation

The more you automate, the harder the leftover human job becomes.

THE PRINCIPLE

Automation doesn't shrink the human role, it transforms it into the hardest parts: passive monitoring plus rare, high-stakes intervention. Worse, by taking over the routine work, automation erodes the very skills and situational feel the operator needs when control is finally handed back. You design away the easy 95% and leave humans the 5% they're now least equipped to handle.

THE TAKEAWAY

Don't just automate the happy path and dump edge cases on a human. Budget design effort for the residual role: keep the operator's context warm and make handback moments rare, clear, and well-supported.

IN PRACTICE

You ship an invoice-processing agent that handles 95% of documents flawlessly, so the AP clerk now just watches a queue and approves the rare exceptions it kicks out. Six months later a malformed multi-currency invoice lands in their lap and they have no idea how to read it: they have not manually processed one since launch, and the agent gives them a half-finished extraction with no context on why it bailed. Do not dump the gnarly 5% on an operator whose skills you have quietly let atrophy. Keep them in the loop on a sample of normal cases too, and when you hand back, hand back the full reasoning trace and a clear statement of exactly what is stuck.

Source: [Ironies of Automation](#) · Lisanne Bainbridge, 1983

Automation Bias

People will trust the machine over their own eyes.

THE PRINCIPLE

Given an automated aid, operators make errors of omission (missing problems it didn't flag) and commission (following its recommendation even when their own valid evidence contradicts it). Automation becomes a heuristic shortcut that replaces vigilant checking, so the agent's recommendation doesn't just inform the human, it overrides their independent judgment.

THE TAKEAWAY

Never present an agent's output as the only signal. Force the human to confront the raw evidence alongside the recommendation, and make disagreement cheap.

IN PRACTICE

Your fraud-review agent flags a transaction as low risk, auto-approve and presents that verdict as a single green badge. The analyst clicks approve without opening the underlying signals, even though the shipping address changed three minutes after a password reset, a pattern they would have caught in a heartbeat on their own. If the recommendation is the only thing on screen, you have built a rubber-stamp machine, not a decision aid. Put the raw evidence next to the verdict, make 'I disagree' a one-click action with no friction, and occasionally withhold the recommendation entirely to keep the human actually looking.

Source: [Does automation bias decision-making?](#) · Skitka, Mosier, Burdick, 1999

Match the Level to the Stakes

Full autonomy is a setting, not a default.

THE PRINCIPLE

Autonomy is a spectrum, from 'the computer suggests' to 'the computer acts then tells you' to 'the computer acts and decides whether to tell you at all'. The highest levels are unwise for consequential actions because no aid is perfectly reliable and the cost of a confident error is unbounded. Autonomy isn't one switch; it's a dial you set per action by how reversible and costly that action is.

THE TAKEAWAY

Don't pick one autonomy level for the whole agent. Gate irreversible or high-impact actions to propose-and-confirm, while letting cheap, reversible ones run fully autonomous.

IN PRACTICE

Your support agent has one autonomy setting: act and report. That is fine when it is resending a receipt, but the same dial lets it issue a \$4,000 refund and cancel an enterprise subscription before anyone sees it. The fix is not a global require-approval flag that buries humans in confirmations for trivial actions, it is gating per action by reversibility and blast radius. Let it resend receipts and reset passwords autonomously, route refunds over a threshold and any cancellation to propose-and-confirm, and you spend human attention only where a confident error actually costs you.

Source: [Human and Computer Control of Undersea Teleoperators](#) • Sheridan & Verplank, 1978

Mind the Mode

Most automation surprises start with 'what mode is it in?'

THE PRINCIPLE

Flexible, multi-mode automation produces 'automation surprises', the system does something unexpected because the operator lost track of which mode it was in, what it would do next, and why. As autonomy grows, the human's job shifts to tracking its state, and every hidden mode transition becomes a latent failure path. An agent that silently changes how it behaves leaves its supervisor one step from being wrong about it.

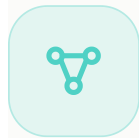
THE TAKEAWAY

Make the agent's current mode, active constraints, and next intended action continuously visible, and never let it switch mode silently. Loud, legible state beats a clever agent the human can't predict.

IN PRACTICE

Your coding agent silently switches from plan mode to auto-apply edits after a tool result, and the developer, still thinking it is drafting a proposal, watches it rewrite twelve files and run a migration. The surprise is not that it acted, it is that nobody knew which mode it was in or what it would do next. An agent that changes how it behaves without announcing it leaves its supervisor one step from being wrong about it. Render the current mode, the active guardrails, and the next intended action somewhere always visible, and make every mode transition an explicit, loud event the human has to see.

Source: [How in the World Did We Ever Get into That Mode?](#) · Sarter & Woods, 1995



PART 10

Trust & Coordination

Working with humans and other agents.

Laws 46 to 50

The Handoff Is the Hard Part

In multi-agent systems, failures live in the seams.

THE PRINCIPLE

Each agent can be flawless in isolation and the system still breaks, because the bug lives between them: what got passed, what got dropped, who owned the state. Sub-agents don't inherit context automatically; anything not explicitly handed over simply doesn't exist on the other side.

THE TAKEAWAY

Design the contract at every boundary. Pass everything the next agent needs explicitly, make state ownership unambiguous, and validate what crosses the seam instead of assuming it survived.

IN PRACTICE

Your orchestrator spawns a research sub-agent and a writer sub-agent, each flawless in isolation, yet the final report cites a competitor's pricing the user never asked about. The bug lives in the seam: the orchestrator passed the topic but dropped the user's 'EU market only' constraint, and the writer had no way to know it ever existed. Sub-agents do not inherit context by osmosis; anything you do not explicitly pass simply does not exist on the other side. Define the contract at every boundary, hand over the full constraint set and source set deliberately, and validate what crosses instead of trusting it survived the trip.

Source: [How we built our multi-agent research system](#) · Anthropic, 2025

Trust Is Calibrated, Not Granted

Autonomy is earned in proportion to track record.

THE PRINCIPLE

People extend an agent freedom the way they extend it to a new hire, incrementally, on reversible things first, widening the leash only as it proves itself. Both failure modes are real: over-trust causes misuse, under-trust causes a good capability to be abandoned. Reliance tracks the perceived reliability the system reveals, not just its true reliability.

THE TAKEAWAY

Start the agent on low-stakes, reversible actions and expand its blast radius as reliability is proven. Show why it's confident where it's strong and flag where it's weak, so users lean on it exactly where they should.

IN PRACTICE

Two failure modes, both expensive. On day one you give the agent direct write access to production billing and it confidently double-applies a discount rule across 800 accounts. Or, burned by that, you wire every single action through manual approval, the team drowns in confirmation fatigue, and within a month they have quietly stopped using a genuinely capable tool. Calibrate instead of swinging between extremes: start it on reversible, low-stakes actions, widen the leash as its track record proves out, and surface where it is reliable versus where it is guessing so people lean on it exactly where they should and not an inch further.

Source: [Trust in Automation: Designing for Appropriate Reliance](#) · Lee & See, 2004

The Escape Hatch Law

No clean exit means a fabricated one.

THE PRINCIPLE

An agent with no legitimate way to say 'I'm stuck' or 'hand this to a human' will invent a path instead. Cornered without an exit, or forced to fill a required field it has no answer for, it fabricates something plausible rather than admitting the gap. A confident hallucination is the default when honesty isn't an option.

THE TAKEAWAY

Always give the agent a first-class way out: an 'escalate to human' action, a nullable field, an explicit 'unknown'. Make 'I don't know' a valid, easy answer and you trade fabrications for honest gaps.

IN PRACTICE

Your intake agent has a required `customer_id` field and no way to signal it could not find one, so when a query arrives with no match it confidently invents a plausible-looking ID and pipes a ticket into the wrong account's history. Cornered without a clean exit, a model fabricates rather than admits the gap; the hallucination is the default, not the anomaly. Give it a first-class way out: a nullable field, an explicit unknown enum, an escalate-to-human tool it is encouraged to call. When 'I do not know' is a valid, easy answer, you trade confident fabrications for honest gaps you can actually act on.

Source: [Reduce hallucinations \(Anthropic Docs\)](#) · [Anthropic](#)

Don't Let the Author Be the Judge

The thing that made it shouldn't grade it.

THE PRINCIPLE

Without an external signal, a model largely fails to self-correct its own reasoning, and often makes correct answers worse by second-guessing them. The model that produced a flawed plan is the same one judging it, with the same blind spots. Real correction needs an outside signal: a tool result, a test that runs, a different model. 'Reflect and try again' on the same model with no new information is theater.

THE TAKEAWAY

Separate generation from judgment. Use an independent instance, fresh context, no memory of the original reasoning, or an external check like a passing test, before trusting a 'corrected' answer.

IN PRACTICE

Your agent writes a SQL query, you prompt it to review your work and fix any bugs, and it cheerfully second-guesses a correct join into a broken one, because it is grading its own reasoning with the exact same blind spots that produced it. Reflection on the same model with no new information is theater: the author cannot see what it could not see the first time. Real correction needs an outside signal. Run the query against a test database, lint it, or hand it to a fresh instance with no memory of the original attempt, and only trust the fixed version once an external check actually passed.

Source: [Large Language Models Cannot Self-Correct Reasoning Yet](#) · [Huang et al., 2023](#)

Preserve Provenance

Don't lose where a fact came from.

THE PRINCIPLE

When findings get summarized and re-summarized, the claim survives but its source, its date, and its uncertainty quietly fall away, until you're holding an assertion you can't verify or defend. Two sources disagreeing isn't noise to flatten; it's signal to keep. A fact without provenance is a rumor with good posture.

THE TAKEAWAY

Carry attribution through every transformation: claim, source, date, confidence. Preserve conflicts with both sides attributed instead of silently picking a winner, so downstream can audit, weigh, and trust.

IN PRACTICE

A research agent reads a 2021 blog post and a 2024 official filing, summarizes both into 'revenue is around \$40M', and three hops of re-summarization later your final report states that figure as flat fact with no date, no source, and no hint that the two inputs actually disagreed. A claim without provenance is a rumor with good posture: you cannot defend it, audit it, or weigh it. Carry the full tuple through every transformation, claim plus source plus date plus confidence, and when sources conflict, keep both with attribution instead of silently crowning a winner. The disagreement is signal, not noise to flatten away.

Source: [Effective Context Engineering for AI Agents](#) · Anthropic

Further reading

The thinking these laws lean on. Foundational essays, papers, and docs worth your time.

01 **Building Effective Agents** · Anthropic Engineering

<https://www.anthropic.com/research/building-effective-agents>

The foundational map of agent patterns, when a workflow beats an agent, and when to add complexity at all. Underpins Narrow Beats General, Determinism at the Edges, and Don't Build an Agent When a Workflow Will Do.

02 **How We Built Our Multi-Agent Research System** · Anthropic Engineering

<https://www.anthropic.com/engineering/built-multi-agent-research-system>

Coordinator / sub-agent design, the 15x token tax, and why the hard bugs live in the handoffs. Backs The Handoff Is the Hard Part and The Multi-Agent Tax.

03 **Effective Context Engineering for AI Agents** · Anthropic Engineering

<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

Curating the context window, what to keep, what to drop, and why more context often hurts. Backs Context Decay and Preserve Provenance.

04 **Writing Tools for Agents** · Anthropic Engineering

<https://www.anthropic.com/engineering/writing-tools-for-agents>

Why tool descriptions are the real interface the model reasons over. Backs The Tool Description Is the Prompt.

05 **Lost in the Middle: How Language Models Use Long Contexts** · Liu et al., 2023

<https://arxiv.org/abs/2307.03172>

The empirical basis for Position Is Power, models reliably use the start and end of long inputs and lose the middle.

06 **The Bitter Lesson** · Richard Sutton, 2019

<http://www.incompleteideas.net/IncIdeas/BitterLesson.html>

Seventy years of AI distilled: general methods plus compute beat hand-crafted cleverness. Backs Stop Tuning, Start Scaling.

07 **Chain-of-Thought & Self-Consistency** · Wei et al. 2022 / Wang et al. 2022

<https://arxiv.org/abs/2201.11903>

Reasoning emerges when you ask for it, and sampling many paths to vote beats one greedy chain. Backs Think Before You Touch and Don't Bet on One Chain.

08 **Retrieval-Augmented Generation (RAG)** · Lewis et al., 2020

<https://arxiv.org/abs/2005.11401>

The original RAG paper, retrieval supplies the facts the generator reasons over. Backs Retrieval Is the Ceiling.

09 **MemGPT: Towards LLMs as Operating Systems** · Packer et al., 2023

<https://arxiv.org/abs/2310.08560>

Treats the context window as RAM and pages memory in and out. Backs Memory Is a System, Not a Window.

- 10 **Your AI Product Needs Evals** · Hamel Husain, 2024
<https://hamel.dev/blog/posts/evals/>
The case for evals as the central discipline of building with LLMs. Backs Vibes Don't Scale and Averages Lie.
- 11 **Judging LLM-as-a-Judge (MT-Bench)** · Zheng et al., 2023
<https://arxiv.org/abs/2306.05685>
Position, verbosity, and self-enhancement biases in LLM graders, with mitigations. Backs The Judge Is Biased.
- 12 **The Lethal Trifecta for AI Agents** · Simon Willison, 2025
<https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
Private data + untrusted content + exfiltration = exploitable. The defining agent-security heuristic. Backs The Lethal Trifecta and Quarantine Untrusted Tokens.
- 13 **OWASP Top 10 for LLM Applications** · OWASP Gen AI Security Project, 2025
<https://genai.owasp.org/llm-top-10/>
The industry-standard catalog of LLM application risks and mitigations. Backs Sandbox the Blast Radius and the safety laws.
- 14 **FrugalGPT: Reducing LLM Cost** · Chen, Zaharia, Zou, 2023
<https://arxiv.org/abs/2305.05176>
Model cascades match top-tier quality at a fraction of the cost. Backs Cascade Before You Escalate.
- 15 **Release It! / CircuitBreaker** · Nygard 2007 / Fowler 2014
<https://martinfowler.com/bliki/CircuitBreaker.html>
Distributed-systems resilience patterns, circuit breakers, bulkheads, fail-fast. Backs Trip the Breaker.
- 16 **Ironies of Automation** · Lisanne Bainbridge, 1983
[https://doi.org/10.1016/0005-1098\(83\)90046-8](https://doi.org/10.1016/0005-1098(83)90046-8)
The foundational human-factors paper: automating the easy work leaves humans the hardest residual role. Backs The Ironies of Automation.
- 17 **Trust in Automation: Designing for Appropriate Reliance** · Lee & See, 2004
https://journals.sagepub.com/doi/10.1518/hfes.46.1.50_30392
The two-sided model of trust, misuse from over-trust, disuse from under-trust. Backs Trust Is Calibrated, Not Granted.
- 18 **Laws of UX** · Jon Yablonski
<https://lawsofux.com>
The format that inspired this deck: durable principles, one card each, named and memorable.

Build agents people actually trust.

These fifty laws are the short version. The real work is applying them under pressure, on your own stack, when the demo meets production. Keep this guide close and revisit it the next time something breaks in a way you have seen before.

Read the [living deck at laws.deleg8.dev](https://laws.deleg8.dev)

New laws are added as they earn their place.

Laws of AI Agents · The Expanded Field Guide · by Sabir Salah · 2026. Inspired by the format of Laws of UX.